

Day 31 — Mon 5 Oct

Countdown: 5 days to CS Degree Paper-I

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

🎯 Degree — algorithm analysis & greedy

Source: ComputerScience_Degree/CSD_01_AlgorithmsAndAnalysis.md

4.3 The Master Theorem — the exam tool

Form: $T(n) = a \cdot T(n/b) + f(n)$, where $a \geq 1$, $b > 1$ are constants and $f(n)$ is asymptotically positive.

Interpretation: the problem is split into a subproblems, each of size n/b , and $f(n)$ is the cost of splitting plus the cost of combining.

Compute the **critical exponent** and the **watershed function**:

$e = \log_b a$ and $n^e = n^{(\log_b a)}$

Then compare $f(n)$ against n^e :

Case	Condition	Result	Reading
1	$f(n) = O(n^{(e-\epsilon)})$ for some $\epsilon > 0$ — <i>f is polynomially smaller</i>	$T(n) = \Theta(n^e)$	The leaves dominate ; the recursion does most of the work
2	$f(n) = \Theta(n^e)$ — <i>they are the same</i>	$T(n) = \Theta(n^e \cdot \log n)$	Balanced ; every level costs the same, and there are $\log n$ levels
3	$f(n) = \Omega(n^{(e+\epsilon)})$ for some $\epsilon > 0$ — <i>f is polynomially larger</i> — AND the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ holds for some $c < 1$ and large n	$T(n) = \Theta(f(n))$	The root dominates ; the top-level work swamps the recursion

The intuition in one sentence: compare the work done at the *leaves* ($n^e \log_b a$ of them) with the work done at the *root* ($f(n)$); whichever is polynomially bigger wins, and if they tie, you pay that amount at each of the $\log n$ levels.

The gap. The Master Theorem does not cover every recurrence. If $f(n)$ is bigger than n^e but not *polynomially* bigger (e.g. only by a log factor), you fall into the gap between Case 2 and Case 3 and must use a recursion tree instead. Say this if you spot one — examiners reward it.

Worked examples — do all of these

(a) Merge sort: $T(n) = 2T(n/2) + n$
 $a = 2, b = 2, f(n) = n. e = \log_2 2 = 1$, so $n^e = n^1 = n$.
 $f(n) = n = \Theta(n^1) = \Theta(n^e) \rightarrow$ **Case 2.**
 $T(n) = \Theta(n \log n)$.

(b) Binary search: $T(n) = T(n/2) + 1$
 $a = 1, b = 2, f(n) = 1. e = \log_2 1 = 0$, so $n^e = n^0 = 1$.
 $f(n) = 1 = \Theta(1) = \Theta(n^e) \rightarrow$ **Case 2.**
 $T(n) = \Theta(\log n)$.

(c) Naive matrix multiplication (divide & conquer): $T(n) = 8T(n/2) + n^2$
 $a = 8, b = 2, f(n) = n^2. e = \log_2 8 = 3$, so $n^e = n^3$.

Is $n^2 = O(n^{(3-\epsilon)})$? Yes, with $\epsilon = 1$ ($n^2 = O(n^2)$) → **Case 1**.

$T(n) = \Theta(n^3)$. (No better than the schoolbook triple loop — which is the point of Strassen.)

(d) Strassen's algorithm: $T(n) = 7T(n/2) + n^2$

$a = 7, b = 2, f(n) = n^2, e = \log_2 7 \approx 2.807$.

Is $n^2 = O(n^{(2.807-\epsilon)})$? Yes, e.g. $\epsilon = 0.3$ → **Case 1**.

$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$. The saving over $\Theta(n^3)$ is exactly the one multiplication saved.

(e) $T(n) = 2T(n/2) + n^2$

$a = 2, b = 2, e = \log_2 2 = 1, n^e = n$.

Is $n^2 = \Omega(n^{(1+\epsilon)})$? Yes, $\epsilon = 1$. Regularity: $a \cdot f(n/b) = 2 \cdot (n/2)^2 = n^2/2 \leq c \cdot n^2$ with $c = 1/2 < 1$ ✓.

→ **Case 3. $T(n) = \Theta(n^2)$** .

(f) $T(n) = 4T(n/2) + n^2$

$e = \log_2 4 = 2, n^e = n^2, f(n) = n^2 = \Theta(n^2)$ → **Case 2. $T(n) = \Theta(n^2 \log n)$** .

(g) $T(n) = 3T(n/4) + n \log n$

$e = \log_4 3 \approx 0.793$. Is $n \log n = \Omega(n^{(0.793+\epsilon)})$? Yes, with $\epsilon = 0.2$ ($n \log n$ grows faster than $n^{0.993}$).

Regularity: $3 \cdot (n/4) \cdot \log(n/4) \leq (3/4) \cdot n \log n$ ✓ with $c = 3/4 < 1$ → **Case 3**.

$T(n) = \Theta(n \log n)$.

(h) $T(n) = 9T(n/3) + n$

$e = \log_3 9 = 2, n^e = n^2, f(n) = n = O(n^{(2-\epsilon)})$ with $\epsilon = 1$ → **Case 1. $T(n) = \Theta(n^2)$** .

(i) A gap case: $T(n) = 2T(n/2) + n \log n$

$e = 1, n^e = n, f(n) = n \log n$ is bigger than n , but $n \log n / n^{(1+\epsilon)} \rightarrow 0$ for every $\epsilon > 0$, so it is **not polynomially** bigger. Not Case 3; not Case 2 ($n \log n \neq \Theta(n)$); not Case 1.

Master Theorem does not apply. By recursion tree, each of the $\log n$ levels costs $\Theta(n \log n)/\dots$ — carefully: level k has 2^k nodes of size $n/2^k$, each costing $(n/2^k) \log(n/2^k)$, so level cost is $n \cdot \log(n/2^k) = n(\log n - k)$. Summing $k = 0$ to $\log n$ gives $n \cdot \sum (\log n - k) = n \cdot (\log^2 n)/2 = \Theta(n \log^2 n)$.

Summary table to memorise:

Recurrence	a	b	$e = \log_b a$	$f(n)$	Case	$T(n)$
$T(n) = T(n/2) + 1$	1	2	0	1	2	$\Theta(\log n)$
$T(n) = 2T(n/2) + 1$	2	2	1	1	1	$\Theta(n)$
$T(n) = 2T(n/2) + n$	2	2	1	n	2	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n^2$	2	2	1	n^2	3	$\Theta(n^2)$
$T(n) = 4T(n/2) + n$	4	2	2	n	1	$\Theta(n^2)$
$T(n) = 4T(n/2) + n^2$	4	2	2	n^2	2	$\Theta(n^2 \log n)$
$T(n) = 4T(n/2) + n^3$	4	2	2	n^3	3	$\Theta(n^3)$
$T(n) = 7T(n/2) + n^2$	7	2	2.807	n^2	1	$\Theta(n^{2.807})$
$T(n) = 8T(n/2) + n^2$	8	2	3	n^2	1	$\Theta(n^3)$
$T(n) = 3T(n/4) + n \log n$	3	4	0.793	$n \log n$	3	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n \log n$	2	2	1	$n \log n$	gap	$\Theta(n \log^2 n)$

4.4 Subtract-and-conquer recurrences

The Master Theorem above is for **dividing** (n/b). When the subproblem shrinks by *subtraction* ($n - b$), use this separate rule:

$T(n) = a \cdot T(n - b) + f(n)$ where $f(n) = O(n^k)$:

Condition	Result
$a < 1$	$T(n) = O(n^k)$
$a = 1$	$T(n) = O(n^{k+1})$
$a > 1$	$T(n) = O(n^k \cdot a^{n/b})$ — exponential

Recurrence	Solution	Where it comes from
$T(n) = T(n-1) + 1$	$\Theta(n)$	Linear search recursively; factorial
$T(n) = T(n-1) + n$	$\Theta(n^2)$	Quick sort worst case ; selection sort
$T(n) = T(n-1) + \log n$	$\Theta(n \log n)$	
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	Towers of Hanoi
$T(n) = 2T(n-1) + n$	$\Theta(2^n)$	
$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(\phi^n)$, $\phi \approx 1.618$	Naive recursive Fibonacci
$T(n) = n \cdot T(n-1) + 1$	$\Theta(n!)$	Permutation generation

5.5 Other divide-and-conquer algorithms worth naming

Algorithm	Recurrence	Complexity
Finding max and min together	$T(n) = 2T(n/2) + 2$	$\Theta(n)$ with $3n/2 - 2$ comparisons (vs $2n - 2$ naively)
Binary exponentiation a^n	$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$
Closest pair of points	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Karatsuba integer multiplication	$T(n) = 3T(n/2) + \Theta(n)$	$\Theta(n^{\log_2 3}) = \Theta(n^{1.585})$
Fast Fourier Transform	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Quickselect (k-th smallest)	$T(n) = T(n/2) + \Theta(n)$ avg	$\Theta(n)$ average, $\Theta(n^2)$ worst
Tower of Hanoi	$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$, exactly $2^n - 1$ moves

When is greedy provably correct?

This is the part examiners look for, and the part students skip. A greedy algorithm is correct if and only if the problem has **both** of these properties:

Property	Statement	Meaning
Greedy-choice property	A globally optimal solution can be reached by making a locally optimal (greedy) choice at each step — you never have to look at the subproblem's solutions to know the first choice is safe	The first greedy choice is <i>safe</i> : some optimal solution contains it
Optimal substructure	An optimal solution to the problem contains within it optimal solutions to its subproblems	After the greedy choice, what remains is the same problem, smaller

How you actually prove it — the exchange argument. Assume an optimal solution OPT that differs from the greedy solution G. Find the first point where they differ. Show that you can *exchange*

OPT's choice for greedy's choice without making OPT worse. Repeat; you transform OPT into G without ever losing value, so G is optimal too. ■

Note the shared property. Optimal substructure is required by *both* greedy and dynamic programming. What separates them is the **greedy-choice property**: greedy has it and can commit immediately; DP does not have it and must therefore evaluate all the options and choose after the fact. **That is the one-sentence answer to "greedy vs DP".**

Formal underpinning (mention it, do not develop it): the class of problems where greedy is guaranteed optimal is exactly the class of **matroids** — a set system closed under subsets and satisfying the exchange property. Prim's, Kruskal's and scheduling-to-minimise-lateness are all matroid problems.

6.3 Huffman coding — work this one fully

The problem. Store a file of characters using as few bits as possible. A **fixed-length** code gives every character the same number of bits, which wastes space when some characters are far more common than others. A **variable-length** code gives frequent characters short codes and rare characters long ones.

The catch: with variable-length codes, how does a decoder know where one code ends? The answer is a **prefix code** (properly, a *prefix-free* code): no codeword is a prefix of any other. Then decoding is unambiguous with no separators — read bits until they match a codeword, emit, repeat.

Huffman's algorithm builds the optimal prefix code, greedily, from the bottom up:

```
HUFFMAN(C)                                // C = set of characters with frequencies
  Q = min-priority queue of all characters, keyed by frequency
  for i = 1 to |C| - 1:
    z = new node
    z.left = x = EXTRACT-MIN(Q)
    z.right = y = EXTRACT-MIN(Q)
    z.freq = x.freq + y.freq
    INSERT(Q, z)
  return EXTRACT-MIN(Q)                    // the single remaining node is the root
```

Greedy criterion: repeatedly merge the two lowest-frequency nodes. The intuition: the two rarest symbols should sit deepest in the tree, so make them siblings at the bottom and treat their combination as a single new symbol.

Worked example

Character	a	b	c	d	e	f
Frequency	45	13	12	16	9	5

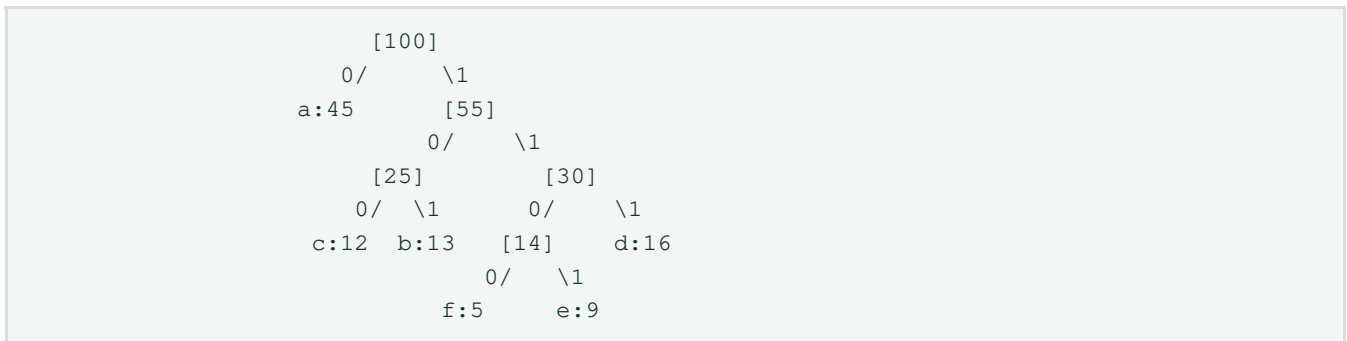
Total = 100 characters.

Trace the merges. Queue is always kept sorted by frequency.

Step	Queue before	Two smallest extracted	New node	Queue after
1	f:5, e:9, c:12, b:13, d:16, a:45	f:5, e:9	(fe):14	c:12, b:13, 14, d:16, a:45
2	c:12, b:13, 14, d:16, a:45	c:12, b:13	(cb):25	14, d:16, 25, a:45
3	14, d:16, 25, a:45	14, d:16	(14,d):30	25, 30, a:45
4	25, 30, a:45	25, 30	(25,30):55	a:45, 55

5	a:45, 55	a:45, 55	root:100	root:100
---	----------	----------	----------	----------

The tree — draw it exactly like this. (Convention: left branch = 0, right branch = 1.)



Read the codes off the tree (the path from the root):

Char	Freq	Code	Length	Freq × Length
a	45	0	1	45
b	13	101	3	39
c	12	100	3	36
d	16	111	3	48
e	9	1101	4	36
f	5	1100	4	20
	100			224 bits

Compare with a fixed-length code. Six characters need $\lceil \log_2 6 \rceil = 3$ bits each $\rightarrow 100 \times 3 = 300$ bits.

Saving = $(300 - 224)/300 = 25.3\%$.

Average codeword length = $224/100 = 2.24$ bits per character.

Verify the prefix property: 0, 101, 100, 111, 1101, 1100 — no codeword is a prefix of another.

Decode 101111100 $\rightarrow$ 101(b) 111(d) 100(c) $\rightarrow$ "bdc". Unambiguous. ✓

Complexity. Building the initial heap $\Theta(n)$; $n-1$ merges each doing two EXTRACT-MINs and one INSERT at $\Theta(\log n) \rightarrow \Theta(n \log n)$. With a pre-sorted input and two queues it drops to $\Theta(n)$.

Properties to state for full marks:

- Huffman produces an **optimal prefix code** — no other prefix code has a smaller expected length.
- It is a **full binary tree**: every internal node has exactly two children. With n characters the tree has n leaves and $n-1$ internal nodes.
- Codes are **not unique** — swapping left/right at any node gives a different but equally optimal code, and ties in the priority queue can be broken either way. **Always state your tie-breaking and 0/1 convention.**
- The most frequent character gets the shortest code; the two least frequent characters always have the **same** (maximum) length and are siblings.
- Used in DEFLATE (ZIP, gzip, PNG), JPEG and MP3 entropy coding.
- Shannon's source coding theorem gives the floor: the entropy $H = -\sum p_i \log_2 p_i$. Huffman is within 1 bit per symbol of it.

6.6 Other greedy algorithms worth naming

Problem	Greedy criterion	Optimal?
---------	------------------	----------

Job sequencing with deadlines	Highest profit first, schedule as late as possible before its deadline	Yes
Minimising average completion time	Shortest job first	Yes
Coin change, canonical systems (₹1,2,5,10,20,50...)	Largest coin $\leq$ remainder	Yes for canonical systems, no in general
Graph colouring (Welsh–Powell)	Highest-degree vertex first	No — heuristic only
Bin packing (first-fit decreasing)	Largest item into the first bin that fits	No — but within $11/9 \cdot \text{OPT} + 1$
Travelling salesman (nearest neighbour)	Nearest unvisited city	No — can be arbitrarily bad
Huffman coding	Merge two lowest frequencies	Yes
Fractional knapsack	Highest value/weight	Yes
0/1 knapsack	(any)	No — needs DP

7.6 Dynamic programming vs divide and conquer — the exam table

	Divide and Conquer	Dynamic Programming
Subproblems	Independent / disjoint — they do not share sub-subproblems	Overlapping — the same subproblem recurs many times
Each subproblem solved	Once, by definition (they're distinct)	Once because it is stored ; recomputation is what DP eliminates
Direction	Top-down recursion	Typically bottom-up tabulation (or top-down with memoisation)
Storage	No table; only the recursion stack	A table of subproblem results
Requires	Optimal substructure	Optimal substructure + overlapping subproblems
Recombination	Combine subproblem answers (merge, etc.)	Choose the best among subproblem answers
Typical complexity	Often $\Theta(n \log n)$	Often $\Theta(n^2)$ or $\Theta(n^3)$ — but converts exponential to polynomial
Space	Usually small	Usually large (the table)
Examples	Binary search, merge sort, quick sort, Strassen, FFT, closest pair	0/1 knapsack, LCS, matrix chain, Floyd–Warshall, Bellman–Ford, optimal BST, edit distance, Fibonacci
If applied to the other's problem	D&C on Fibonacci → exponential blow-up from recomputation	DP on merge sort → wasted table, no benefit, since no subproblem repeats

The one-line answer: *both* split a problem into subproblems, but divide and conquer's subproblems are disjoint so recursion suffices, whereas dynamic programming's subproblems overlap, so it stores results to avoid exponential recomputation.

7.7 Greedy vs dynamic programming

	Greedy	Dynamic Programming
Decision making	Makes one choice at each step and never revisits it	Considers all choices at each step and picks the best
Direction	Top-down: choose first, then solve the subproblem	Bottom-up: solve subproblems first, then choose
Requires	Optimal substructure + greedy-choice property	Optimal substructure + overlapping subproblems
Optimality	Only for problems with the greedy-choice property	Always optimal when the recurrence is correct
Speed	Faster — usually $\Theta(n \log n)$	Slower — usually $\Theta(n^2)/\Theta(n^3)$
Space	$\Theta(1)$ extra typically	Table required
The classic contrast	Fractional knapsack: $\Theta(n \log n)$	0/1 knapsack: $\Theta(nW)$

🕒 Forensic — Windows artifacts

Source: ComputerForensic/FOR_02_SystemArtifacts.md

5.1 The Windows Registry — concept

Intuition first

Before Windows 95, configuration lived in scattered `.INI` text files. The **registry** replaced them with a single **hierarchical, transactional database** of configuration for the machine, the hardware, the installed software and each user. Because Windows and almost every application write their settings there — including "convenience" settings like *which files you opened most recently* and *which USB stick you plugged in* — **the registry is an unintentional activity log**.

Structure mirrors a file system:

```
HIVE → KEY → SUBKEY → VALUE (name, type, data)
                                types: REG_SZ, REG_EXPAND_SZ, REG_BINARY,
                                      REG_DWORD, REG_QWORD, REG_MULTI_SZ
```

Every KEY carries a LastWrite timestamp (UTC). ← this is what makes the registry a timeline source, not just a settings store.

Critical exam point: registry *keys* have a LastWrite time; registry *values* do not.

So when an artifact stores one item per key (e.g. each USB device gets its own key), you get a usable timestamp; when many items share one key (e.g. RunMRU), the LastWrite reflects only the most recent change.

The five root keys (hives as seen in `regedit`)

Root key	Abbrev.	Contents
<code>HKEY_LOCAL_MACHINE</code>	HKLM	Machine-wide settings: hardware, installed software, services, security
<code>HKEY_USERS</code>	HKU	Loaded profiles of all logged-on users, keyed by SID

HKEY_CURRENT_USER	HKCU	A <i>link</i> to the current user's subtree under HKU
HKEY_CLASSES_ROOT	HKCR	A <i>merged view</i> of HKLM\SOFTWARE\Classes and HKCU\Software\Classes — file associations, COM
HKEY_CURRENT_CONFIG	HKCC	A link to the current hardware profile in HKLM\SYSTEM\CurrentControlSet\Hardware Profiles\Current

Only HKLM and HKU are "real"; HKCU, HKCR and HKCC are **derived/volatile links**. This is a classic MCQ.

5.2 Registry hive files and their on-disk locations — memorise this table

Logical hive	On-disk file	Contents
HKLM\SYSTEM	%SystemRoot%\System32\config\SYSTEM	Services, drivers, USBSTOR , MountedDevice , time zone, computer name, ControlSets
HKLM\SOFTWARE	%SystemRoot%\System32\config\SOFTWARE	Installed applications, OS version and install date, NetworkList , Rur keys, profile list
HKLM\SAM	%SystemRoot%\System32\config\SAM	Local user accounts , RIDs, group membership, password hashes, last-logon/last-password-change
HKLM\SECURITY	%SystemRoot%\System32\config\SECURITY	Local security policy, LSA secrets , cached domain credential
HKLM\HARDWARE	(none — built in RAM at every boot)	Detected hardware. Volatile: only obtainable from a live system or a memory image

HKU\<SID> (per user)	C:\Users\<user>\NTUSER.DAT	The user's own settings: RecentDocs, RunMRU, TypedPaths, UserAssist, WordWheelQuery MountPoints2, mapped drives
HKU\<SID>_Classes (per user)	C:\Users\<user>\AppData\Local\Microsoft\Windows\UsrClass.dat	ShellBags (Windows 7+) , per-user COM/file associations
HKU\.DEFAULT	%SystemRoot%\System32\config\DEFAULT	Settings for the LocalSystem context / pre-logged on
(older / XP)	C:\Documents and Settings\<user>\NTUSER.DAT	Same as above on XP

Supporting files: each hive has .LOG, .LOG1, .LOG2 transaction logs in the same folder, and older systems had .SAV copies. **Always collect the .LOG files with the hive** — modern tools replay them to recover the most recent, not-yet-flushed changes and sometimes deleted keys.

Backups and previous versions:

- %SystemRoot%\System32\config\RegBack\ — periodic backup copies (⚠ note: automatic RegBack was disabled by default from Windows 10 version 1803 onward, though it can be re-enabled).
- **Volume Shadow Copies** contain older hive versions — often the only place a deleted key survives.
- %SystemRoot%\repair\ on older systems.

Deleted registry data: deleted keys/values are marked unallocated inside the hive file but the cells often remain; tools such as RegRipper/Registry Explorer/yarp can recover them. **Registry slack is a real and examinable concept.**

5.3 Key forensic registry keys — the master table

This is the single most memorisation-heavy table in the whole elective, and it is worth doing. A 10- or 15-mark "discuss the forensic value of the Windows registry" answer that reproduces even eight of these rows with the right hive will score at or near full marks.

(a) System and configuration

Purpose	Key
Computer name	SYSTEM\CurrentControlSet\Control\ComputerName\ComputerName
Time zone (essential for interpreting all timestamps)	SYSTEM\CurrentControlSet\Control\TimeZoneInformation
Which ControlSet is current	SYSTEM\Select → Current, LastKnownGood
OS version, build, install date , registered owner	SOFTWARE\Microsoft\Windows NT\CurrentVersion
Last shutdown time	SYSTEM\CurrentControlSet\Control\Windows → ShutdownTime (binary FILETIME)

Installed applications	SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall
Services and drivers	SYSTEM\CurrentControlSet\Services
User profile list (SID → username → profile path)	SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList
Whether prefetch is enabled	SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\PrefetchParameters
Whether last-access timestamps are disabled	same FileSystem key → NtfsDisableLastAccessUpdate (disabled by default since Vista — an important caveat when interpreting access times)
ClearPageFileAtShutdown (anti-forensic setting)	SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management

(b) USB device history — a guaranteed question

What you learn	Key / file
Every USB mass-storage device ever attached: vendor, product, revision and serial number ; each device is its own subkey, so the key LastWrite ≈ first/last connection	SYSTEM\CurrentControlSet\Enum\USBSTOR\Disk&Ven_X&Prod_Y&Rev_Z\<SerialNumber>
All USB devices including non-storage (VID/PID)	SYSTEM\CurrentControlSet\Enum\USB\VID_xxxx&PID_yyyy\<serial>
First install, last connected, last removed timestamps per device	SYSTEM\CurrentControlSet\Enum\USBSTOR\... \Properties\{83da6326-97a6-4088-9453-a1923f573b29}\0064 / 0066 / 0067 (⚠ verify GUID digits before quoting; the concept — that per-device install/first-connect/last-connect/last-removal timestamps live under Properties — is the examinable part)
Volume GUID / drive-letter mapping for the device	SYSTEM\MountedDevices (\DosDevices\E: and \??\Volume{GUID} entries, whose data contains the device signature)

Which user used the device	NTUSER.DAT → Software\Microsoft\Windows\CurrentVersion\Explorer\MountPoints2\{Volume GUID} — matching the GUID from MountedDevices attributes the device to a specific user account
Human-readable device names	SYSTEM\CurrentControlSet\Enum\USBSTOR FriendlyName, and SOFTWARE\Microsoft\Windows Portable Devices\Devices (holds the volume label)
First-ever installation time, driver install log	C:\Windows\INF\setupapi.dev.log (setupapi.log on XP) — text file, easy to read, gives the first connection date/time
Live/removal events	Event log Microsoft-Windows-DriverFrameworks-UserMode\Operational (IDs 2003, 2100, 2102) and Partition/Diagnostic (ID 1006) △ verify IDs

The classic 10-mark walkthrough — "prove that a specific USB stick was connected by a specific user":

1. USBSTOR → find the device by vendor/product; record the **serial number**.
2. USBSTOR\...\Properties → first-install and last-connected timestamps.
3. setupapi.dev.log → corroborate the **first** connection date.
4. MountedDevices → find the **Volume GUID** and the drive letter assigned.
5. NTUSER.DAT\...\MountPoints2\{that GUID} in a **specific user's** hive → attributes it to **that user**; the key LastWrite gives the time.
6. Windows Portable Devices\Devices → volume label, to match the physical exhibit.
7. Corroborate with **LNK files and Jump Lists** whose target path is on that volume (they store the volume serial number and label), and with **ShellBags** for folders browsed on it.
8. Corroborate with the **event log** entries above.

Notice the shape of that answer: *artifact* → *what it proves* → *corroboration*. Use that shape for every "how would you prove X" question in this paper.

(c) MRU (Most Recently Used) lists — user activity

All of these live in NTUSER.DAT under

Software\Microsoft\Windows\CurrentVersion\Explorer\:

Key	What it records
RecentDocs	Recently opened documents , sub-keyed by file extension; holds an MRUListEx ordering
RunMRU	Everything typed into the Start → Run dialog (a favourite of intruders using cmd, \\server\share)
TypedPaths	Paths typed into the Explorer address bar
ComDlg32\OpenSavePidlMRU	Files chosen in Open/Save common dialogs, grouped by extension
ComDlg32\LastVisitedPidlMRU	The application used and the folder it last used for Open/Save
WordWheelQuery	Terms typed into the Windows Explorer / Start-menu search box — shows what the user was <i>looking for</i> , often the most probative artifact of all
RecentApps (Win10-era)	Recently used applications and the files they opened △ verify presence per build

Map Network Drive MRU	Network shares mapped by the user
TypedURLs (under Software\Microsoft\Internet Explorer\)	URLs typed into IE/legacy Edge — typed, not merely visited

Application MRUs are equally valuable: Software\Microsoft\Office\<ver>\<App>\File MRU, media-player recent files, archive-tool recent paths, and so on.

(d) Run keys / persistence — malware and autostart

Key	Behaviour
SOFTWARE\Microsoft\Windows\CurrentVersion\Run	Runs at every logon (all users)
SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce	Runs once, then the value is deleted
NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Run / RunOnce	Same, per user
...\Policies\Explorer\Run	Policy-driven autostart
SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon → Shell, Userinit	Hijacked by malware — should be explorer userinit.exe respectively; anything appended suspicious
SYSTEM\CurrentControlSet\Services	Services and drivers with Start=2 (auto) or Start=0 (boot/system driver)
SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\<exe> → Debugger	Classic IFEO hijack — launches an attacker binary when the named exe runs (also the "sticky keys" backdoor)
AppInit_DLLs, AppCertDlls, KnownDLLs	DLL injection persistence
Startup folders (not registry)	%AppData%\Microsoft\Windows\Start Menu\Programs\Startup (per user); %ProgramData%\Microsoft\Windows\Start Menu\Programs\Startup (all users)
Scheduled Tasks (not registry)	C:\Windows\System32\Tasks\ (XML files) SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks
WMI event subscriptions	C:\Windows\System32\wbem\Repository\Classes__SystemEvents — fileless persistence

(e) UserAssist — proof of GUI program execution

- **Location:** NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{GUID}\Count
- **What it records:** programs the user launched **through the Windows GUI** (Explorer, Start menu, desktop shortcuts), with a **run count**, a **last-execution timestamp** and (on later versions) a focus-time value.
- **The famous quirk:** value names are **ROT-13 encoded** (each letter rotated 13 places) — purely as light obfuscation. Tools decode automatically. *This single fact appears in MCQ sets constantly.*
- **Limitation to state in the answer:** UserAssist records **GUI-launched** programs only.

Anything started from a command line, a script, a service or a scheduled task **will not appear** — so UserAssist absence is not evidence of non-execution. Corroborate with Prefetch, ShimCache and Amcache.

- Common GUIDs: {CEBFF5CD-...} for executables and {F4E57C4B-...} for shortcut links
 ▲ verify the full GUIDs before writing them out; naming the *concept* suffices.

(f) ShellBags — proof that a folder was browsed

- **Concept:** Windows remembers, per user, the **view settings** (icon size, sort order, column layout, window position) of every folder you open in Explorer — so the folder looks the same next time. To do that it must record **that the folder was opened**, and it keeps that record **even after the folder is deleted or the removable drive is gone**.
- **Location:**
 - Windows 7 and later: `UsrClass.dat` → `Local Settings\Software\Microsoft\Windows\Shell\BagMRU` and `...\Shell\Bags`
 - Also `NTUSER.DAT\Software\Microsoft\Windows\Shell\BagMRU / Bags`
 - Windows XP: `NTUSER.DAT\Software\Microsoft\Windows\ShellNoRoam\BagMRU`
- **Forensic value — say all four:**
 1. Proves a **specific user** browsed a **specific folder path**, with a timestamp.
 2. **Survives deletion** of the folder and disconnection of the device — evidence of folders on **removable media, network shares, encrypted containers (TrueCrypt/VeraCrypt volumes) and ZIP archives** that no longer exist or are not present.
 3. Reconstructs a **directory tree** the examiner has never seen, from names alone (useful when the content is gone but the folder names — "Invoices_2025", "backup_of_client_db" — are themselves probative).
 4. The `BagMRU` structure preserves the **hierarchy** and MRU **ordering** of browsing.
- **Limitation:** ShellBags record folders opened in **Explorer/dialog windows**; folders touched only by the command line generally do not appear.

(g) Network and miscellaneous

Purpose	Key
Networks the machine joined (SSID/domain, first and last connected time , network type)	<code>SOFTWARE\Microsoft\Windows NT\CurrentVersion\NetworkList\Profiles\{GUID}</code> and <code>...\Signatures\Unmanaged</code> (holds the gateway MAC , which geolocates the connection)
Static/DHCP IP, DHCP server, lease time per interface	<code>SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces\{GUID}</code>
ShimCache / AppCompatCache (evidence of executables present on the system, with file path, size and last-modified time; presence, not necessarily execution)	<code>SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache</code>

Amcache (installed/executed programs, SHA-1 hash of the binary , path, publisher) — a <i>file</i> , not part of the standard hives	<code>C:\Windows\AppCompat\Programs\Amcache.hve</code>
MUICache (application names of executed programs)	<code>UsrClass.dat\Local Settings\...\MuiCache</code>
BAM/DAM (Background/Desktop Activity Moderator) — last execution time per executable per user SID	<code>SYSTEM\CurrentControlSet\Services\bam\State\UserSettings\<SID></code> ⚠ verify path per build
SRUM (System Resource Usage Monitor — per-application network bytes sent/received , useful for proving data exfiltration)	<code>C:\Windows\System32\SRU\SRUDB.dat</code> (ESE database, not registry)
Recently mounted/known volumes	<code>SYSTEM\MountedDevices</code>
Autologon credentials (sometimes plaintext!)	<code>SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon</code> → <code>DefaultUserName</code> , <code>DefaultPassword</code>

The "program execution" family is a favourite comparison question. Learn the distinction:

Artifact	Proves	Gives run count?	Gives timestamps?
Prefetch	Execution	✓	Last 8 run times + first run
UserAssist	GUI execution by that user	✓	Last run
ShimCache	Presence (and possibly execution)	✗	File last-modified; cache entry order
Amcache	Presence/installation + SHA-1 hash	✗	First-seen
BAM/DAM	Execution per user	✗	Last run
SRUM	Execution + network bytes	—	Hourly buckets
Event 4688	Process creation (if auditing enabled)	✗	Exact time + command line

5.4 Windows Event Logs

Concept

Windows records system, security and application events in structured logs. Since Vista these are `.evtx` files (XML-based binary); XP and earlier used `.evt`.

Location: C:\Windows\System32\winevt\Logs*.evtx

(XP: C:\Windows\System32\config*.evt)

The three classic logs, plus the modern additions:

Log	Contents
Security.evtx	Logons, logoffs, privilege use, object access, policy change, account management — only what auditing is configured to record
System.evtx	Driver/service events, boot and shutdown, hardware, time changes
Application.evtx	Application-generated events, crashes
Setup.evtx	Servicing/updates
Microsoft-Windows-TerminalServices-*/Operational	RDP connections (source IP, username)
Microsoft-Windows-Windows Defender/Operational	Malware detections
Microsoft-Windows-PowerShell/Operational (4103, 4104)	Script block logging — the actual PowerShell commands run
Microsoft-Windows-TaskScheduler/Operational	Task creation/execution
Microsoft-Windows-DriverFrameworks-UserMode/Operational	USB insert/remove
Microsoft-Windows-Bits-Client/Operational	BITS transfers — used for exfiltration

Key event IDs — memorise the starred ones

ID	Log	Meaning
4624 ★	Security	Successful logon (check the Logon Type)
4625 ★	Security	Failed logon — repeated 4625s = brute force
4634 / 4647	Security	Logoff / user-initiated logoff
4648	Security	Logon using explicit credentials (runas) — lateral movement indicator
4672	Security	Special privileges assigned to a new logon (administrative logon)
4720 / 4726	Security	User account created / deleted
4722 / 4725	Security	Account enabled / disabled
4724 / 4723	Security	Password reset by admin / changed by user
4728 / 4732 / 4756	Security	Member added to a global / local / universal security group (privilege escalation)

4688 ★	Security	New process created (with command line if the policy is enabled)
4689	Security	Process exited
4663 / 4656 / 4660	Security	Object access attempted / handle requested / object deleted (needs SACL auditing)
4698 / 4702	Security	Scheduled task created / updated
4697	Security	Service installed — a classic malware persistence indicator
1102 ★	Security	The audit log was cleared — a top-tier anti-forensic indicator
104	System	An event log was cleared (System log analogue of 1102)
7045 ★	System	A new service was installed
7034 / 7035 / 7036	System	Service crashed / control sent / entered running-stopped state
6005 / 6006	System	Event log service started / stopped — proxy for boot / clean shutdown
6008	System	Unexpected shutdown (crash or power loss)
6013	System	System uptime (daily)
1074	System	Shutdown/restart initiated, with the user and reason
41	System	System rebooted without a clean shutdown (Kernel-Power)
1 (Sysmon)	Sysmon	Process creation with hashes — only if Sysmon is installed
4776	Security	NTLM credential validation
5140 / 5145	Security	Network share accessed / detailed share access
1149	TerminalServices-RemoteConnectionManager	RDP authentication succeeded (gives the source IP)
21 / 24 / 25	TerminalServices-LocalSessionManager	RDP session logon / disconnect / reconnect
20001	System (DriverFrameworks/UMPnP)	Device driver installation — USB attachment

△ Event ID meanings occasionally shift across Windows versions and some IDs above appear in multiple providers. The starred ones are stable and safe to quote. Verify anything unusual against Microsoft documentation rather than memory.

Logon Types in event 4624 — a very common sub-question

Type	Meaning
2	Interactive — logged on at the physical console
3	Network — accessed a share or service over the network (most common)

4	Batch — scheduled task
5	Service — a service started with these credentials
7	Unlock — workstation unlocked
8	NetworkCleartext — credentials sent in the clear (e.g. basic auth)
9	NewCredentials — <code>runas /netonly</code>
10	RemoteInteractive — RDP / Terminal Services
11	CachedInteractive — logged on with cached domain credentials (laptop off-network)

Anti-forensics note: event logs are a prime target for tampering. Watch for **1102/104** (log cleared), for **gaps in the record-number sequence**, for a suspiciously small `.evtx` file, and for a log whose earliest entry post-dates the incident. Deleted event records can often be carved from unallocated space, the pagefile and Volume Shadow Copies.

5.7 Alternate Data Streams (ADS) — a guaranteed topic

The intuition

On NTFS, a file's content is stored in a `$DATA` attribute. The **default, unnamed `$DATA` stream** is what you see when you open the file. But NTFS allows **any number of additional, named `$DATA` streams attached to the same file**. Windows Explorer shows only the unnamed stream, and **the file's reported size counts only the unnamed stream**. So you can hide an entire executable inside a 0-byte text file and Explorer will insist the file is 0 bytes.

Historically ADS existed for **Macintosh interoperability** — HFS files had a data fork and a **resource fork**, and NTFS needed somewhere to put the resource fork when serving files to Mac clients via Services for Macintosh.

Syntax

```
filename:streamname:$DATA      ← the general form

C:\> echo secret text > report.txt:hidden.txt
C:\> type nc.exe > innocent.doc:nc.exe
C:\> dir                        ← report.txt shows 0 bytes; the stream is invisible
C:\> dir /R                     ← Vista+ : LISTS THE STREAMS
C:\> more < report.txt:hidden.txt
PS> Get-Item report.txt -Stream *      ← PowerShell
PS> Get-Content report.txt -Stream hidden.txt
C:\> wmic process call create "C:\path\innocent.doc:nc.exe" ← older execution trick
```

Legitimate uses (mention these — it shows understanding)

Stream	Purpose
<code>:Zone.Identifier</code>	The Mark of the Web — written by browsers and mail clients to record that a file came from the internet. Contains <code>ZoneId=3</code> (Internet), and on modern Windows also the <code>HostUrl</code> and <code>ReferrerUrl</code> . Forensically superb: it proves a file was downloaded, and from where
<code>:favicon</code>	Site icons

Term	Precise meaning	What it contains
RAM slack (a.k.a. sector slack / memory slack)	The space from the end of the file to the end of the last partially used sector	Historically: whatever happened to be in RAM at the moment of the write — hence the name; potentially passwords, fragments of other documents, memory structures. Modern Windows/Linux pad this with zeros , so it is far less productive today than the textbooks suggest — state both facts
Drive slack (a.k.a. file slack proper, cluster slack)	The completely unused sectors between the end of the last used sector and the end of the last cluster	Data from a previously deleted file that occupied those sectors — never overwritten because the OS only wrote as far as the new file's length
File slack	RAM slack + drive slack — the whole gap between end-of-file and end-of-last-cluster	Both of the above
Volume slack (a.k.a. partition slack / drive slack in some texts)	The space between the end of the file system (the last cluster the file system uses) and the end of the partition/ volume itself — arising because the partition size is rarely an exact multiple of the cluster size, or because a volume was resized	Data from a previous, larger file system on that partition — including old file-system metadata, and it is a favourite place to hide data because no file system references it at all
<i>(related)</i> Partition/unpartitioned space	Space on the physical disk not assigned to any partition — e.g. before the first partition, between partitions, or after the last	Old partitions, hidden data, boot code

The comparison table to reproduce

	File slack	Volume slack
Where	Inside an allocated cluster belonging to a live file	Between the end of the file system and the end of the partition
Boundary it arises from	File size vs cluster size	File-system size vs partition size
Owned by	A specific existing file	No file, and no file system
Typical size	0 to (cluster size – 1) per file; huge in aggregate across thousands of files	A few sectors up to many megabytes, once per volume
Typical content	Remnants of previously deleted files; historically RAM contents	Remnants of a previous/larger file system; deliberately hidden data
Visible to the OS?	No — the OS reads only up to the file's logical length	No — outside the file system entirely
How recovered	Forensic tool extracts the bytes beyond logical EOF for every file; keyword-search the slack	Image the whole partition (and the whole physical disk) and examine the tail region

Anti-forensic use	Small payloads hidden in slack (e.g. <code>slacker.exe</code> -style tools)	Larger payloads hidden where no file system will ever touch them
--------------------------	---	--

Related: unallocated space

- **Definition:** clusters that the file system's bitmap/FAT marks as **free** — either never used or belonging to **deleted files**.
- **Why it matters:** this is where the *bodies* of deleted files sit until overwritten. It is the primary target of **file carving** (FOR_03 §6.3).
- **Distinguish clearly in the exam:**
- **File slack** = space *inside an allocated cluster* that the live file does not use.
- **Unallocated space** = *whole clusters* not allocated to any live file.
- **Volume slack** = space *inside the partition but outside the file system*.
- **Unpartitioned space** = space *on the disk but outside every partition*.

Practical exam sentence: "A bit-stream (physical) image captures file slack, volume slack, unallocated space and unpartitioned space; a logical copy captures none of them. This is the whole reason forensic imaging must be physical."

Likely exam framing

- 5 marks: "Differentiate between file slack and volume slack." → the table above + the diagram.
- 10 marks: add RAM vs drive slack, unallocated space, and why physical imaging is required.

5.11 Thumbnail cache (thumbcache) and other image artifacts

Artifact	Location	Value
<code>Thumbs.db</code> (XP / Win 2000, and still generated for network folders)	In each folder browsed in thumbnail view	Contains thumbnails of images in the folder. Becomes a file in the folder, easy to find and delete.

Thumbcache (Vista+)	C: <code>\Users\<user>\AppData\Local\Microsoft\Windows\Explorer\thumbcache_*.db</code> (sizes 32, 96, 256, 1024, 1280, 1920 ...) plus <code>thumbcache_idx.db</code>	Central thumbnail cache for all folders. Provides preview icons and thumbnails. Look for evergreen original deleted unrecovered — deleted CSA images case
Icon cache	<code>iconcache_*.db</code> in the same folder	Icon instance execution application evidence now program exists
WebCacheV01.dat	<code>%LocalAppData%\Microsoft\Windows\WebCache\</code>	ESE holds Edge, cook, cache, meta, user, Win, com
.DS_Store equivalent	—	(mac see

Note the pattern: **caches are built for performance, and performance caches never get cleaned up properly.** That is why they are the examiner's friend.

§5.13 Windows artifacts — the single consolidated location table

If you memorise nothing else in §5, memorise this. It is the answer to "List the important Windows forensic artifacts and their locations."

Artifact	Location
Registry hives (machine)	<code>C:\Windows\System32\config\</code> — SYSTEM, SOFTWARE, SAM, SECURITY, DEFAULT (+ <code>.LOG*</code>)
Registry hive (per user)	<code>C:\Users\<u>\NTUSER.DAT</code>

Registry hive (per user, ShellBags)	C:\Users\<u>\AppData\Local\Microsoft\Windows\UsrClass.dat
Event logs	C:\Windows\System32\winevt\Logs*.evtx
Prefetch	C:\Windows\Prefetch*.pf
Amcache	C:\Windows\AppCompat\Programs\Amcache.hve
SRUM	C:\Windows\System32\SRU\SRUDB.dat
Recent LNK files	C:\Users\<u>\AppData\Roaming\Microsoft\Windows\Recent\
Jump Lists	...\Recent\AutomaticDestinations\ and ...\CustomDestinations\
Recycle Bin	C:\\$Recycle.Bin\<SID>\\$I..., \$R...
Thumbnail cache	C:\Users\<u>\AppData\Local\Microsoft\Windows\Explorer\thumbcache_*.db
Scheduled tasks	C:\Windows\System32\Tasks\
Startup folder	C:\Users\<u>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\
Hosts file (DNS hijack)	C:\Windows\System32\drivers\etc\hosts
USB driver install log	C:\Windows\INF\setupapi.dev.log
Pagefile / swapfile / hiberfil	C:\pagefile.sys, C:\swapfile.sys, C:\hiberfil.sys
Volume Shadow Copies	\System Volume Information\
Windows Search index	C:\ProgramData\Microsoft\Search\Data\Applications\Windows\Windows.edb
Browser data	see FOR_03 §8
WMI repository	C:\Windows\System32\wbem\Repository\OBJECTS.DATA
User profiles	C:\Users\<u>\ (Desktop, Documents, Downloads, AppData{Roaming,Local,LocalLow})

§5.14 Likely exam questions (Windows artifacts)

Marks	Question
5	What is the Windows registry? Name the five root keys.
5	List the registry hive files and their locations on disk.
5	What is an Alternate Data Stream? How is it detected?
5	Differentiate between file slack and volume slack.
5	What is a prefetch file? What forensic information does it provide?
5	Explain the \$I and \$R files of the Windows Recycle Bin.

5	What is UserAssist? Why are its entries ROT-13 encoded?
5	What are ShellBags and what do they prove?
5	State the forensic value of <code>pagefile.sys</code> and <code>hiberfil.sys</code> .
10	Discuss the forensic importance of the Windows registry, listing at least eight keys of evidentiary value.
10	Describe the Windows event log. List ten important event IDs with their meaning.
10	What is a LNK file? Explain how LNK files can prove access to a file on a removable drive that is no longer available.
10	Explain slack space, unallocated space and volume slack with a diagram, and their role in evidence recovery.
15	"Windows records far more than the user realises." Discuss the principal Windows system artifacts, their on-disk locations and the investigative questions each answers.
15	An employee is suspected of copying confidential files to a USB drive and deleting the originals. Describe, artifact by artifact, how you would prove this from a forensic image of his Windows workstation.

MCQ traps (Windows artifacts)

- **NTUSER.DAT is per user; SYSTEM/SOFTWARE/SAM are machine-wide** in `System32\config`.
- **HKEY_LOCAL_MACHINE\HARDWARE has no file on disk** — it is rebuilt at every boot.
- Registry **keys** have LastWrite times; **values** do not.
- **UserAssist = ROT-13**. (Not Base64, not XOR.)
- **Prefetch proves execution; ShimCache proves presence**.
- Prefetch limit: 128 (Win7) → **1024 (Win8+)**; **last 8 run times on Win8+**, only 1 on Win7.
- **ShellBags live mainly in UstrClass.dat** on Windows 7+.
- **\$I holds the metadata, \$R holds the data**. Constantly swapped in options.
- XP used `RECYCLER` + a single `INFO2` file; Vista+ uses `$Recycle.Bin + $I/$R`.
- **ADS is NTFS-only** and is destroyed by copying to FAT.
- **RAM slack is to the end of the SECTOR; drive/file slack is to the end of the CLUSTER**.
- **hiberfil.sys = compressed RAM image; pagefile.sys = virtual memory overflow**.
- **Event 1102 = security log cleared; 4624 = successful logon; 4625 = failed logon; 7045 = new service installed**. These four are the most-swapped.
- Logon **type 10 = RDP, type 2 = interactive console, type 3 = network**.

§6. Linux System and Artifacts

7.5 System logs and user artifacts

Logs

Log	Location	Note

Unified Logging (macOS 10.12+)	<code>/private/var/db/diagnostics/*.*tracev3</code> + <code>/private/var/db/uuidtext/</code>	Binary, compressed, high-volume. Read with <code>log show --predicate ...</code> or forensic parsers (e.g. <code>mandiant/macOS-UnifiedLogs</code>). Replaced most legacy <code>.log</code> files. Retention is short (days to weeks) — collect early
Legacy ASL	<code>/private/var/log/asl/*.*asl</code>	Apple System Log — older systems
Classic Unix logs	<code>/private/var/log/</code> — <code>system.log</code> , <code>install.log</code> , <code>wifi.log</code> , <code>secure.log</code> (older)	<code>install.log</code> is very useful: records every software installation and macOS update with timestamps
Audit trail	<code>/private/var/audit/</code>	BSM audit records, if <code>auditd</code> enabled
Crash/diagnostic reports	<code>~/Library/Logs/DiagnosticReports/</code> , <code>/Library/Logs/DiagnosticReports/</code>	Proves a program ran (it crashed) and often captures its arguments and loaded libraries
Console/user logs	<code>~/Library/Logs/</code>	Per-application logs

The four signature user artifacts

Artifact	Location	What it gives you
Keychain	<code>~/Library/Keychains/login.keychain-db</code> (user), <code>/Library/Keychains/System.keychain</code> (system, incl. Wi-Fi passwords), <code>~/Library/Keychains/<UUID>/keychain-2.db</code> (iCloud/data-protection keychain)	Stored passwords — website logins, Wi-Fi, app credentials, certificates, secure notes. Encrypted with the user's login password : if you have the password (or the user's session is live) the contents are extractable (<code>security dump-keychain -d</code>). Even without the password, the metadata — service names, account names, creation and modification dates — reveals which accounts and services the user held
Spotlight	<code>/.Spotlight-V100/Store-V2/<UUID>/store.db</code> (and <code>.store.db</code>); per-volume, including external drives	The system-wide metadata index of every file — names, paths, sizes, MIME types, and content-derived metadata , plus <code>kMDItemUsedDates</code> / last-used and last-download dates . Crucially it retains entries for files that have been deleted , letting you prove a document with a given name and size existed. Query a live system with <code>mdfind</code> , inspect a file's metadata with <code>mdls</code>
FSEvents	<code>/.fsevents/</code> (per volume, including external and USB volumes)	The file-system change journal used by Time Machine and Spotlight. Records that a path was created, modified, renamed or deleted , with an event ID ordering. It gives you a history of file-system activity including on removable drives — often the only proof that files were copied to a USB device . Note: it stores paths and event flags, not timestamps for every event , so times are approximate/derived from the log-file boundaries

Recents / recent items	<code>~/Library/Application Support/com.apple.sharedfilelist/*.sfl2</code> (Recent Documents, Recent Applications, Recent Servers, Recent Volumes), plus per-app <code>com.apple.recentitems</code> plists	The macOS analogue of Windows RecentDocs/Jump Lists. RecentServers and RecentVolumes prove connections to network shares and external drives
-------------------------------	--	--

Other high-value macOS user artifacts

Artifact	Location
Time Machine backups	<code>/Volumes/<backup>/Backups.backupdb/</code> or APFS snapshots — complete historical copies of the whole system
APFS local snapshots	<code>tmutil listlocalsnapshots /</code> — recoverable prior states even without an external backup drive
Safari history / downloads / bookmarks / cache	<code>~/Library/Safari/History.db</code> , <code>Downloads.plist</code> , <code>Bookmarks.plist</code> , <code>~/Library/Caches/com.apple.Safari/</code> — see FOR_03 §8
Messages / iMessage	<code>~/Library/Messages/chat.db</code> (SQLite) + <code>Attachments/</code> — often the single most probative file on a Mac
Mail	<code>~/Library/Mail/V*/</code> — <code>.emlx</code> message files
Notes, Calendar, Contacts, Reminders	<code>~/Library/Group Containers/...</code> (SQLite)
Photos library	<code>~/Pictures/Photos Library.photoslibrary/database/Photos.sqlite</code>
Quarantine event database	<code>~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2</code> (SQLite) — every file downloaded, with the URL and timestamp △ verify path on recent versions
Installed apps and their first-run	<code>/var/db/SystemPolicyConfiguration/ExecPolicy</code> (SQLite), <code>install.log</code>
Terminal history	<code>~/.bash_history</code> , <code>~/.zsh_history</code> (zsh is the default shell since Catalina)
iOS device backups	<code>~/Library/Application Support/MobileSync/Backup/</code> — a full iPhone backup sitting on the Mac . Always check for this in a mobile case
User account records	<code>/private/var/db/dslocal/nodes/Default/users/*.plist</code> (the macOS equivalent of <code>/etc/passwd+shadow</code> — includes the <code>ShadowHashData</code> password verifier)

9.2 TP-I Unit 8 — Operating Systems, UNIX, Windows OS

→ Covered in `_Shared\CORE_03_OperatingSystems.md` and `_Shared\CORE_04_UnixLinux.md` — revise those.

(OS functions; multiprogramming/multiprocessing/multitasking; virtual memory, paging, fragmentation; mutual exclusion, critical regions, locks; CPU/IO/resource scheduling, deadlock; disk management, formatting, boot block, free-space management, RAID; protection and security, domains, access control, Trojan/virus/worm, authentication; the UNIX file system, process management, shell variables, command-line programming, filters and commands; Windows design principles, system components, Terminal Services, Fast User Switching, file system, networking.)

Plus these forensic-specific angles (most are developed in detail in §5 and §6 above):

OS concept	Forensic slant
File systems	§4, §5.9 — allocation units, file slack , unallocated space, journaling as an evidence source
Deleted-file mechanics	§4.2, §6.2 — deletion changes bookkeeping, not data. FOR_03 §6 for recovery and carving
Virtual memory / paging	pagefile.sys and the swap partition are memory on disk — searchable for keys, passwords and document fragments (§5.12)
Hibernation	hiberfil.sys is a full RAM image on disk — the best offline memory evidence (§5.12)
Process management	Live forensics: process list, parent-child relationships (a winword.exe spawning cmd.exe is anomalous), loaded DLLs, handles, network sockets. Volatility does all of this from a memory image
Scheduling / services	Persistence mechanisms — Services, Scheduled Tasks, cron, systemd timers (§5.3d, §6.8)
Free-space management	The bitmap/FAT is <i>exactly</i> the structure that tells you which clusters are unallocated → the carving target
RAID	Must image every member disk and reconstruct; note the level, stripe size and disk order (§2.3)
Protection, access control, authentication	ACLs and permissions establish who could have done it ; SAM/shadow hashes establish credential compromise ; SUID files are backdoors (§6.3)
Trojan / virus / worm	Malware taxonomy is examined in FOR_03 §2.7
UNIX commands	The examiner's own toolset: dd , dc3dd , md5sum/sha256sum , strings , grep -a , file , xxd/hexdump , stat , lsof , find -perm -4000 , last , lastb , journalctl , mount -o ro,noexec,nodev,noatime,loop and losetup -r for read-only image mounting, fdisk -l , blkid , debugfs , foremost/scalpel for carving
Windows Terminal Services / Fast User Switching	Multiple concurrent user sessions → attribution problem : which session did the act? Resolve with logon type 10 (RDP) events, session IDs, and per-user artifacts
Windows networking	SMB share access events (5140/5145), MountPoints2 , mapped-drive MRU